How the Agent Registry & A2A Communication Work

Deep-dive architectural report detailing agent self-registration, identity resolution, the Google A2A protocol handshake, and production patterns.

1. High-Level Multi-Agent Architecture

In this autonomous scheduling platform, the system is decomposed into a **Company Coordinator Agent** (orchestrating overall scheduling intents) and multiple distributed **Employee Domain Agents** (Priya, Arjun, Meera), each safeguarding an individual's calendar, privacy boundaries, and rules.

2. The Agent Registry: How it Works

The registry acts as the **phonebook and capability broker**. Rather than hardcoding URLs or ports into the company coordinator, agents dynamically publish their endpoints, skills, and availability.

Key Registry Components in Code:

- **File Store:** data/agent_registry.json persists registered agents across restarts.
- **Thread Safety:** Atomic replacement via NamedTemporaryFile, os.fsync, and threading.RLock prevents race conditions and corrupted reads.
- **Dynamic Proxy:** RegistryEmployeesMapping implements Python's MutableMapping interface so legacy dictionary lookups read from the live registry seamlessly.

Step 1: Agent Self-Registration on Startup

When an employee agent starts (e.g. python employee_agent.py priya 9101), it defines its A2A Agent Card and invokes agent_registry.register_agent():

```
# agents/employee_agent.py (Lines 248-262)
card = make_card(f"{name.title()}'s Calendar Agent", ..., port, skill)

# Dynamic registration with the central agent registry
agent_registry.register_agent(
    agent_id=name,                      # "priya"
    url=f"http://127.0.0.1:{port}",      # "http://127.0.0.1:9101"
    name=display_name,                  # "Priya Sharma"
    role="employee",
    skills=["calendar", "meetings", "book_meeting"],
    status="online",
)
serve(card, EmployeeExecutor(name), port)
```

Resulting Registry Record (data/agent_registry.json):

```
{
  "priya": {
    "id": "priya",
    "name": "Priya Sharma",
    "url": "http://127.0.0.1:9101",
    "role": "employee",
    "skills": ["calendar", "meetings", "book_meeting"],
    "status": "online",
    "registered_at": "2026-10-23T00:00:00+05:30",
    "last_seen": "2026-10-24T07:47:18+05:30"
  }
}
```

3. How Company Agent Discovers Agents (Semantic Search vs Deterministic Lookup)

Important Distinction: Is Semantic Search Used Here?

No. In calendar scheduling, attendee identity resolution is **deterministic**. When a user asks to meet with 'Priya', resolving via vector similarity risks fuzzy mismatches (e.g., returning 'Priyanka'). Exact string normalization is used for identity, while semantic search is reserved for open-ended skill discovery.

The Dynamic Dict Proxy (RegistryEmployeesMapping)

In agents/company_tools.py, line 32 sets EMPLOYEES = agent_registry.RegistryEmployeesMapping(). Any lookup like EMPLOYEES['priya'] executes live queries against the registry:

```
# agents/agent_registry.py (Lines 303-349)
class RegistryEmployeesMapping(MutableMapping):
    def __getitem__(self, key):
        urls = get_employee_urls()             # Reads data/agent_registry.json
        norm_key = str(key).strip().lower()
        if norm_key in urls:
            return urls[norm_key]             # Returns "http://127.0.0.1:9101"
        raise KeyError(key)

    def __contains__(self, key):
        return str(key).strip().lower() in get_employee_urls()
```

Validation and Connection in company_tools.py

When the LLM decides to invite attendees, it executes create_meeting or send_invite. The tool validates the attendees against the live registry:

```
# agents/company_tools.py (Lines 495-504)
for invitee in invitees:
    if not isinstance(invitee, str) or invitee.strip().lower() not in EMPLOYEES:
        return f"ERROR: unknown employee '{invitee}'. Known: {', '.join(EMPLOYEES)}"

    employee = invitee.strip().lower()
    normalized_invitees.append(employee)

# Direct connection resolution (Line 965):
client = await connect(EMPLOYEES[employee])  # connect("http://127.0.0.1:9101")
```

4. How Agents Communicate Over A2A (Protocol Handshake)

Communication between agents uses Google's **A2A (Agent-to-Agent) Standard Protocol**. It operates in two distinct phases: *Card Resolution* and *Stateful Streaming Session*.

Phase	Action & Endpoint	Payload / Outcome
1. Card Resolution	GET /.well-known/agent-card.json HTTP GET to employee agent URL	Returns Agent Card JSON: protocol version, capabilities, skills, and authentication schema.
2. Client Init	create_client(card, config) ClientConfig(streaming=True)	Initializes HTTPX streaming client with auth token headers.

3. Message Dispatch	client.send_message() A2A SendMessageRequest	Payload contains meeting JSON, UUID message ID, optional task_id and context_id.
4. Event Stream	Async Server-Sent Events (SSE) task, status_update, artifact_update	Streams execution progress and emits task status (COMPLETED, INPUT_REQUIRED, REJECTED).

Code Behind A2A Handshake (agents/a2a_client.py)

```
# agents/a2a_client.py (Lines 22-57)
async def get_agent_card(url: str):
    async with httpx.AsyncClient(timeout=5, headers=headers) as http:
        # Step 1: Resolves agent card from standard well-known endpoint
        return await A2ACardResolver(http, url).get_agent_card()

async def connect(url: str):
    card = await get_agent_card(url)
    http = httpx.AsyncClient(timeout=30, headers=headers)
    # Step 2: Creates A2A client from discovered card
    return await create_client(card, client_config=ClientConfig(streaming=True, httpx_client=http))

async def call(client, text, task_id=None, context_id=None):
    # Step 3: Transmits message and listens to event stream
    async for ev in client.send_message(SendMessageRequest(
            message=user_message(text, task_id, context_id))):
        if ev.HasField("task"):
            state = TaskState.Name(ev.task.status.state)
        elif ev.HasField("artifact_update"):
            artifact = get_artifact_text(ev.artifact_update.artifact)
    return state, artifact, task_id, context_id
```

5. Stateful Negotiation & Conflict Resolution Flow

A central innovation of the A2A design is that negotiations are not simple fire-and-forget RPCs. They maintain **durable, resumable task contexts**.

The Four Core Task States

TASK_STATE_WORKING	Agent is analyzing rules, checking calendar free/busy slots, or querying external APIs.
TASK_STATE_COMPLETED	Meeting accepted and booked on calendar. Event JSON returned as artifact payload.
TASK_STATE_INPUT_REQUIRED	A clash or policy question was found. The agent pauses execution and yields control to the human employee.
TASK_STATE_REJECTED / FAILED	Meeting was declined due to strict personal rule, existing immovable event, or user refusal.

What happens when Priya has a conflict? (Step-by-Step)

1. Company Agent sends invite for 'Product Review' at 10:30 AM to Priya's agent over A2A.
2. Priya's Agent inspects her calendar, finds an overlapping 'Client Call'.
3. Priya's Agent returns TASK_STATE_INPUT_REQUIRED: *'Client Call overlaps. Reschedule or decline?'*
4. Company Tools record this session in data/tasks.json with a task_id and context_id.
5. Priya logs into her Employee Dashboard, sees the decision card, and selects *'Reschedule client call'*.
6. Dashboard calls reply_to_employee(meeting_id, employee, reply) which sends the answer back to Priya's agent using the exact same task_id and context_id, resuming the original task!

6. Local Prototype vs. Production Enterprise Architecture

In production, while the protocol concepts (Agent Cards, Task States, Tool Loops) stay identical, the infrastructure layers are hardened for massive concurrency, high availability, and zero-trust security.

Architecture Layer	Current Prototype	Enterprise Production Grade
Registry & Discovery	Local JSON file (agent_registry.json) with filesystem atomic locks.	Distributed consensus store (**etcd**, **Consul**, **Redis Cluster**) or Kubernetes CoreDNS service discovery.
Agent Addressing	Localhost static ports (http://127.0.0.1:9101).	Internal Service Mesh URIs (**Envoy / Istio**) with load balancing and circuit breaking.
Security & Auth	Shared static Bearer token.	**mTLS** with SPIFFE/SPIRE machine IDs + **OAuth2 Token Exchange (RFC 8693)** for user-delegated authority.
Skill Discovery	Static skill tag list strings.	**pgvector / Pinecone** vector embeddings of Agent Cards for semantic semantic-skill routing.

Durable Execution	In-memory ThreadPool + JSON task file.	Durable workflow orchestration (**Temporal.io**, AWS Step Functions, Kafka event queues) surviving restarts.

Conclusion: The Power of A2A

By enforcing strict A2A boundaries, agents maintain domain autonomy. The Company Agent never reads or manipulates Priya's raw calendar data directly; it sends an invitation proposal and Priya's personal agent applies her private constraints, rules, and human confirmations before committing.